

Custom Metrics for LLM Eval Scorecard

Goal

Allow users to evaluate LLM outputs using their own custom metrics, not just the standard built-in metrics.

The final output should be a scorecard showing how each model performed against the selected custom metrics.

Requirements

1. Custom metric folder

The project should support a folder where users can add their own Python metric files.

Example folder:

```
custom_metrics/
```

Each file in this folder represents one custom metric.

2. Metric registration

Each custom metric should have a clear name so the system can identify it.

For example:

```
answer_relevance
```

```
format_correctness
```

```
citation_quality
```

3. User request should select metrics

When the user sends an evaluation request, they should specify which metrics they want to run.

Example request:

Evaluate `model_a` and `model_b` using these metrics:

`answer_relevance`

`format_correctness`

The system should only run the metrics requested by the user.

4. Load custom metric code

The project should find the requested metric files from the custom metrics folder and load them.

If a metric does not exist, the system should return a clear error message.

Example:

`Metric "citation_quality" was requested but not found.`

5. Run metrics against model outputs

For each model response, the system should run the selected custom metrics.

Each metric should return a score.

The score can be numeric, such as:

`0 to 1`

or

`0 to 100`

The project should define one standard scoring format so the scorecard is consistent.

6. Generate scorecard

The final output should be a scorecard that shows:

Model name

Metric name

Score for each metric

Overall score

Optional explanation from the metric, if available

7. Handle errors safely

If one custom metric fails, the full evaluation should not crash.

The scorecard should show that the metric failed and include the error reason.

Example

A user creates a custom metric called:

`answer_length_quality`

This metric checks whether the model answer is neither too short nor too long.

The user then sends a request:

Evaluate these models:

`gpt-model-a`

`claude-model-b`

Use this custom metric:

`answer_length_quality`

Expected scorecard:

Model	answer_length_quality	Overall Score
gpt-model-a	85	85
claude-model-b	72	72

In this example, the project loaded the user's custom metric, applied it to each model output, and produced a scorecard based on that custom metric.